

Computer Science

08. Virtual Memory

8주차: 가상메모리

목차 INDEX

01 Virtual Memory

- 가상 메모리 공간
- Page / Frame
- Paging
- Page Table
- Protection

02 Address Translation

- MMU
- TLB
- Physical Address

03 Demand Paging

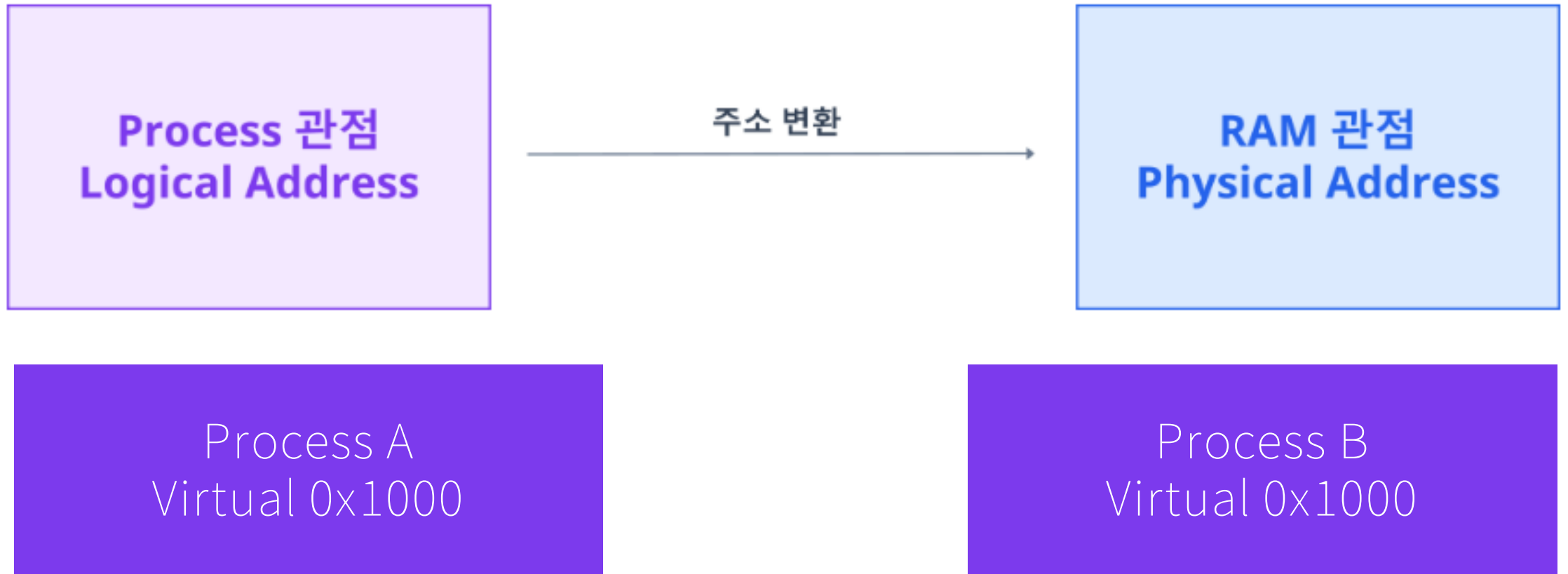
- Page Fault
- Replacement
- FIFO / LRU
- Swap
- Thrashing

01

가상메모리

Virtual Memory

가상메모리



두 프로세스가 같은 주소를 사용한다면?

가상메모리



각 Process에 독립적인 가상메모리 공간 제공

Mapping과 접근 권한 관리, 필요한 Page만 RAM에 배치

Mapping: 가상 메모리의 Page가 실제 RAM의 어느 Frame에 대응되는지

Page / Frame

PAGE

Virtual Address Space의 고정 크기 단위
프로세스는 여러 페이지로 조각나
프레임에 할당됨

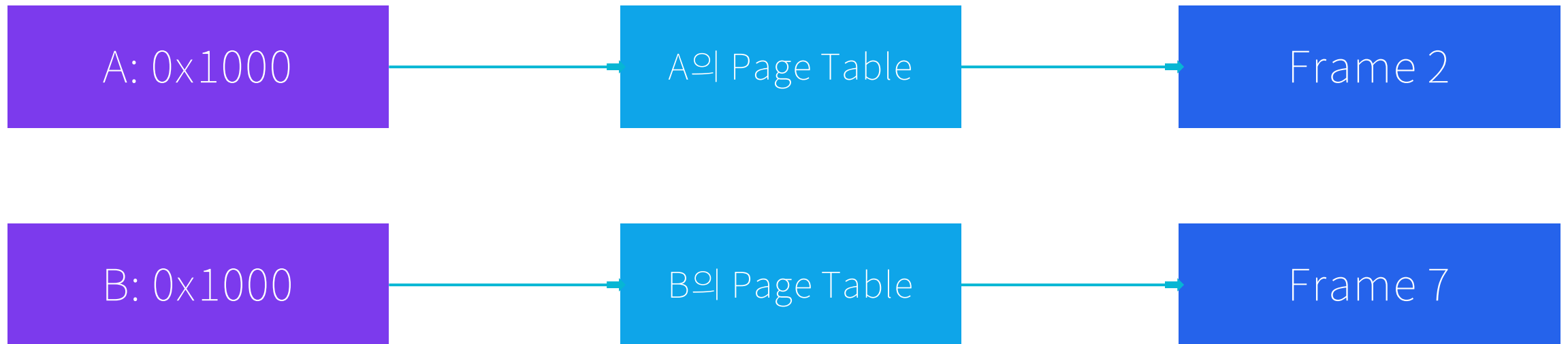
P3

FRAME

Physical Memory의 고정 크기 단위

F6

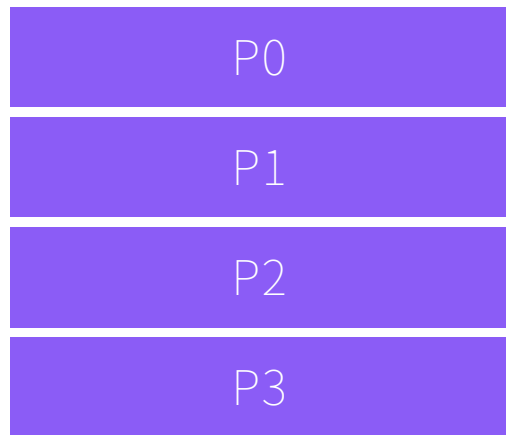
Page Table



Page Table은 Virtual Address를
실제 RAM의 Physical Address로 변환함

Paging

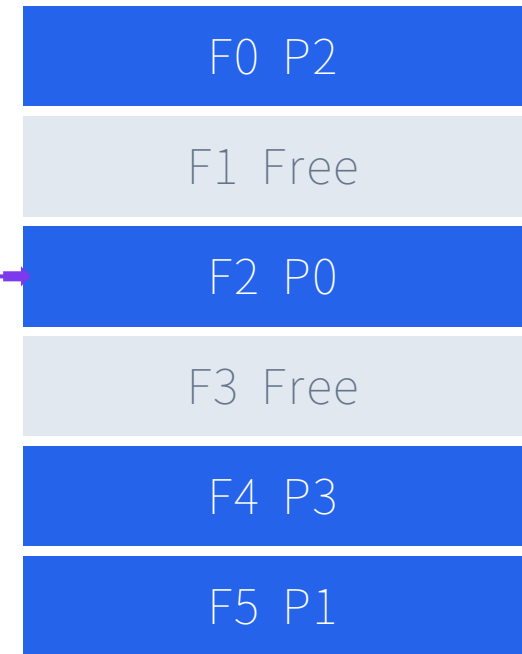
VIRTUAL PAGES



Page Table

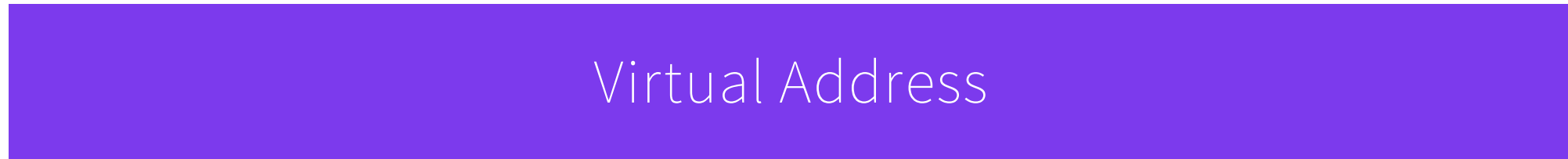
P0 → F2
P1 → F5
P2 → F0
P3 → F4

PHYSICAL FRAMES



Page Table에는 Mapping, 권한, 상태 정보 저장

Virtual Address의 구조



Virtual Page Number(VPN)

어떤 Page인가?

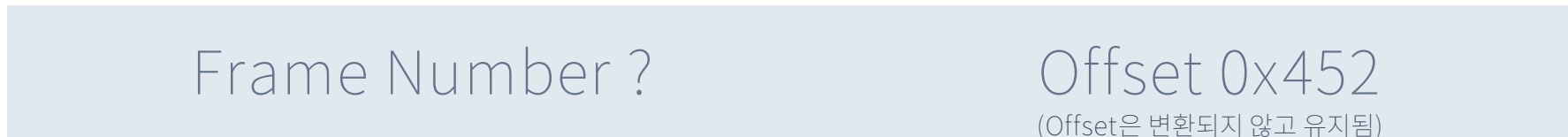
Offset

Page 안에서 어디인가?

CPU는 가상 주소를 읽어 어느 페이지의 어느 데이터에 접근할지를 확인함

Virtual Address → Physical Address 변환

Virtual Address

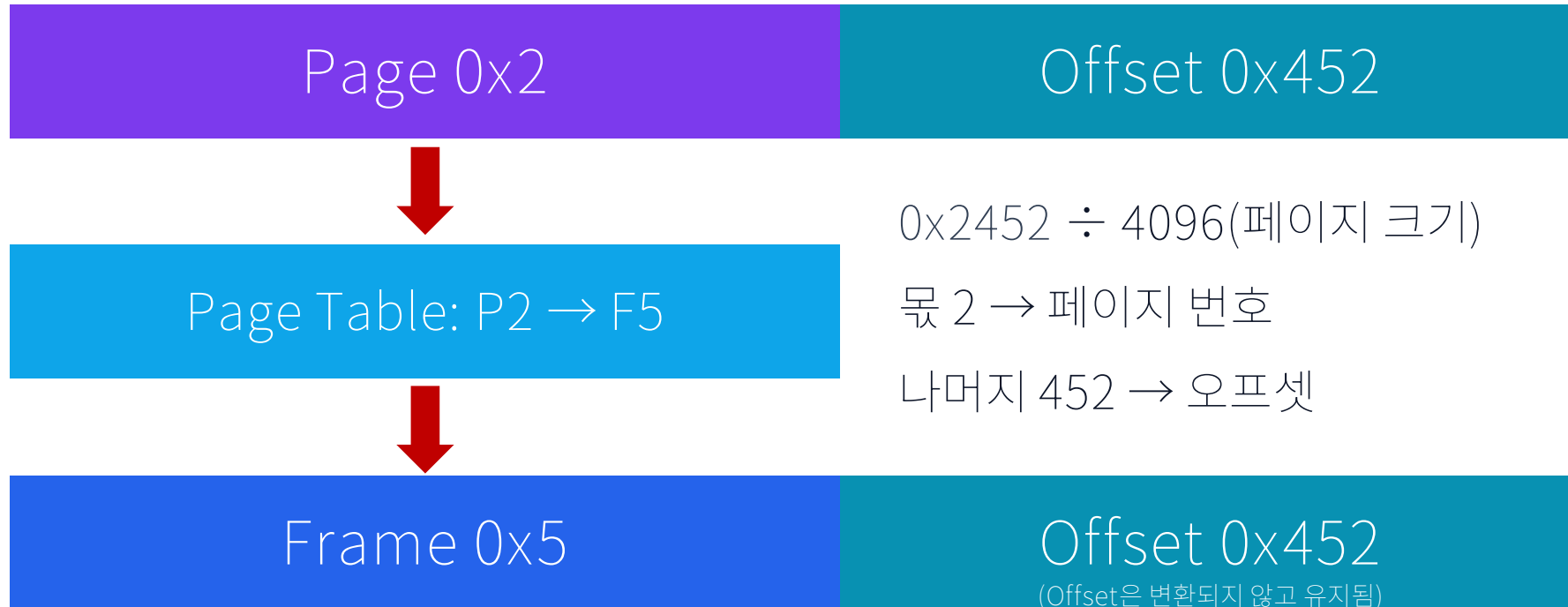


Virtual Address: 0x2452

$$(2 \times 4096 + 0x452 = 0x2452)$$

Virtual Address → Physical Address 변환

Virtual Address



Physical Address: 0x5452

$$(5 \times 4096 + 0x452 = 0x5452)$$

02

주소 변환

Address Translation

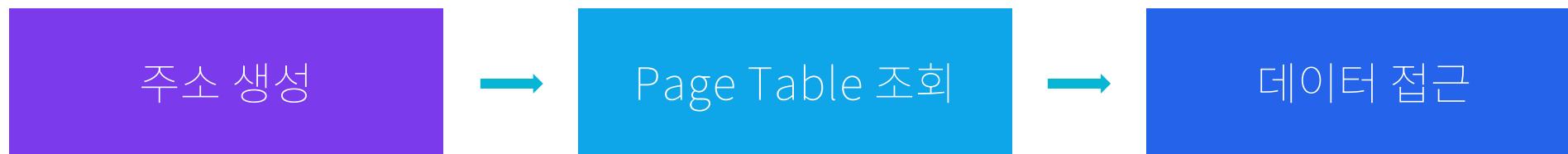
MMU: Memory Management Unit



VPN으로 Page Table 조회

실제 컴퓨터에서는 가상주소를 물리주소로 변환해주는 하드웨어가 있음
그걸 MMU, Memory Management Unit이라 부름

MMU: Memory Management Unit



Page Table은 RAM에 저장되어있음

→ 변환 정보를 읽는 작업(즉, 추가적인 연산시간)이 추가된다

최근 변환 결과를 가까운 곳에 기억하면?

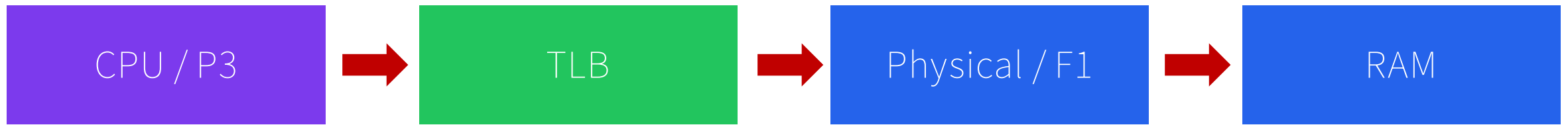
TLB: Translation Lookaside Buffer

Virtual Page	Physical Frame
P0	F5
P3	F1
P7	F4

TLB;
Translation
Lookaside Buffer

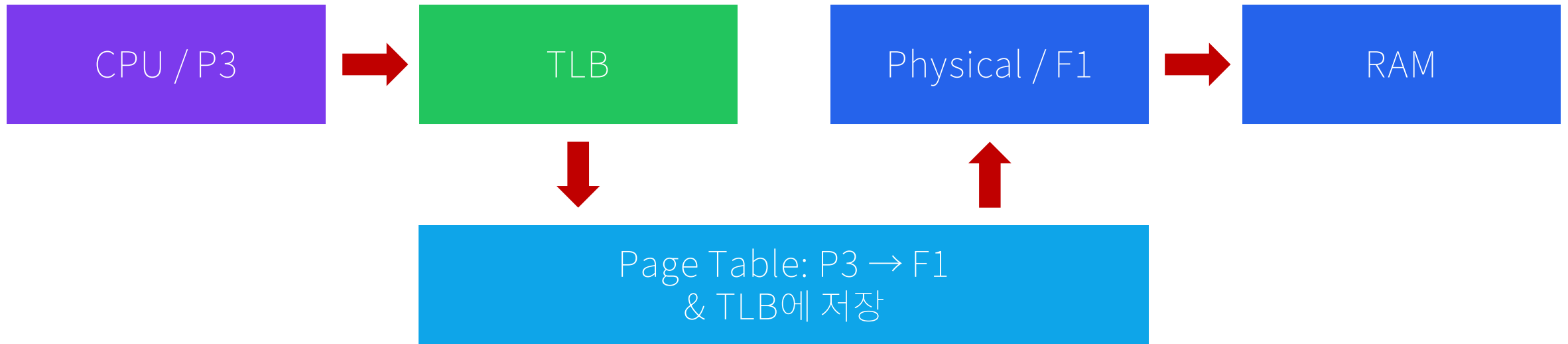
페이지 테이블 중 최근에 사용했거나
유지할만한 일부 정보만 따로 저장하는
일종의 캐시

TLB Hit



만약 현재 찾는 VPN이 TLB에 있다면 결과 즉시 반환
페이지 테이블 전체를 읽는 시간을 아낄 수 있음

TLB Miss



만약 요청하는 주소가 TLB에 없으면 페이지 테이블에서 찾음
이후 해당 변환을 TLB에도 저장함

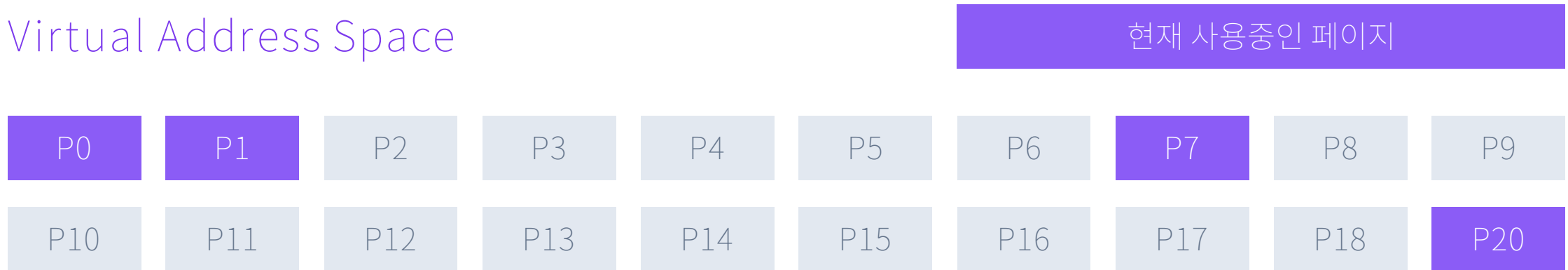
03

요구 페이징

Demand Paging

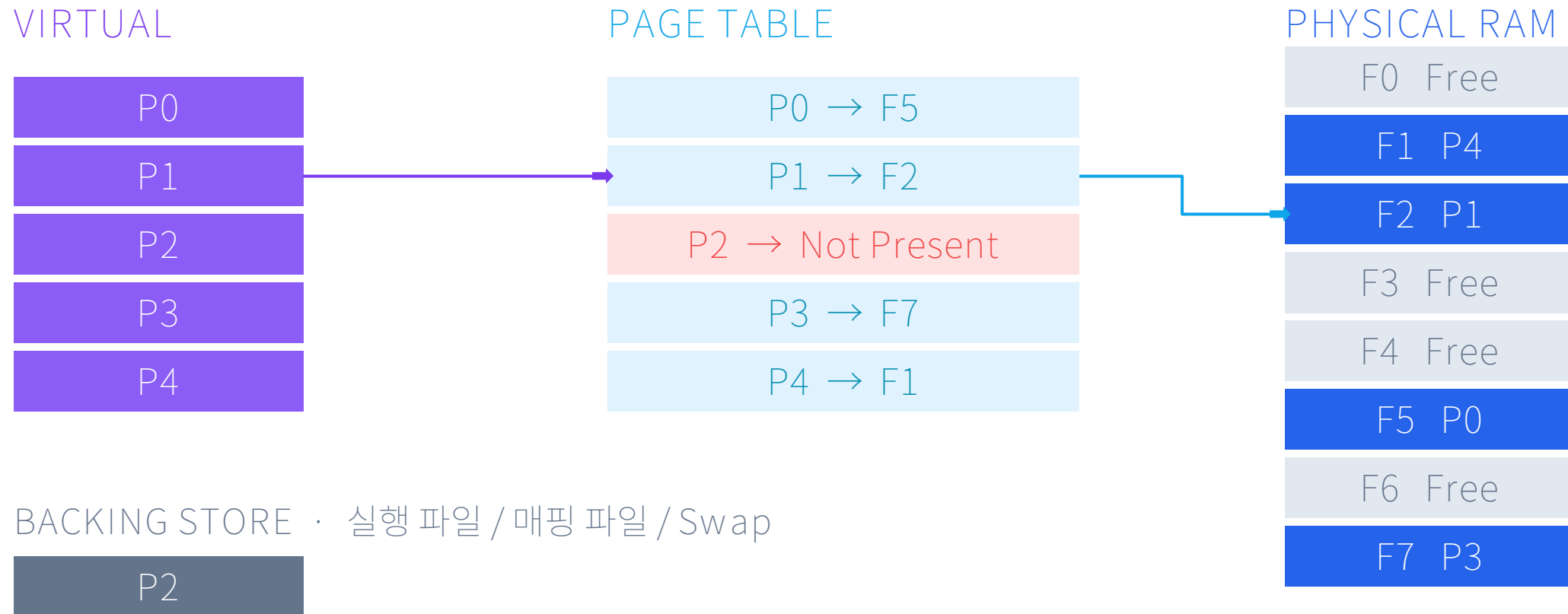
Demand Paging

Virtual Address Space



100개의 페이지를 모두 RAM에 올릴 필요는 없어보임

Demand Paging



만약 CPU가 RAM에 없는 P2에 접근하려 하면?

Present & Not Present

Page	Frame	Is Present?	상태
P1	F2	1: Present	RAM에 있음
P2	—	0: Not Present	RAM에 없음
P3	F5	1: Present	RAM에 있음

CPU가 Not Present인 페이지에 접근하면 Page Fault 발생

Page Fault를 해결하는 과정

VIRTUAL

P0
P1
P2
P3

PAGE TABLE

P0 → F0 / 1
P1 → F1 / 1
P2 → F2 / 1
P3 → — / 0

PHYSICAL RAM

F0 P0
F1 P1
F2 P2
F3 Free

Disk: P3

CPU가 P3 접근 → Present=0 확인

CPU가 P3에 접근: Not Present인 상황

Page Fault 발생

Page Fault를 해결하는 과정

VIRTUAL

P0
P1
P2
P3

PAGE TABLE

P0 → F0 / 1
P1 → F1 / 1
P2 → F2 / 1
P3 → — / 0

PHYSICAL RAM

F0 P0
F1 P1
F2 P2
F3 Free

Disk: P3

OS: 접근 유효성 확인

기존 User Mode에서 프로그램을 실행하다 Page Fault(예외 상황) 발생
Kernal mode로 전환되어 OS의 Page Fault Handler로 Page Fault 해결

사용자모드 & 커널모드 User Mode & Kernel Mode

상단 영역: User Mode (사용자 모드)

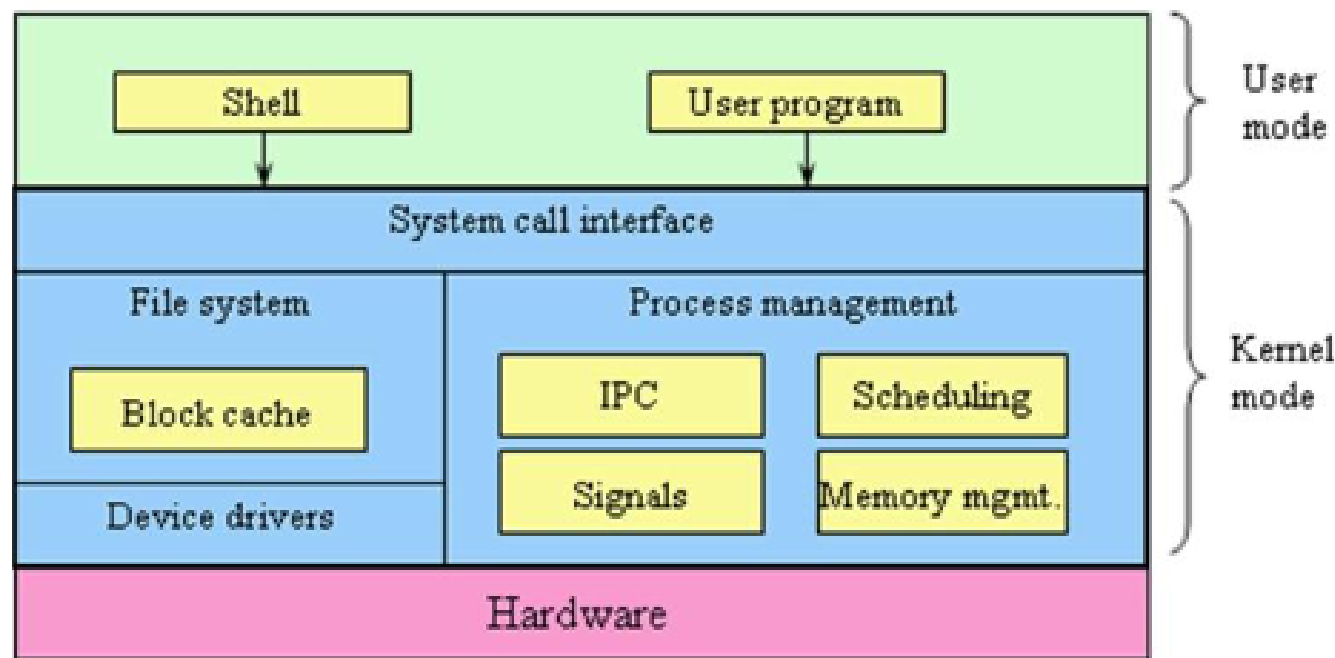
Python 프로그램, 웹 브라우저, 메모장 등 일반 애플리케이션 실행 영역
(하드웨어 직접 접근 불가)

System Call Interface

일반 프로그램이 커널에게 필요한 작업을 요청하는 공식적 통로

하단 영역: Kernel Mode (커널 모드)

운영체제 핵심(커널), 파일 시스템, 메모리 관리, 장치 드라이버
(하드웨어 직접 제어 가능)



일반 프로그램이 위험한 하드웨어 자원을 직접 망가뜨리지 못하도록,
OS가 두 개의 모드로 권한을 엄격히 분리

Page Fault를 해결하는 과정

VIRTUAL

P0
P1
P2
P3

PAGE TABLE

P0 → F0 / 1
P1 → F1 / 1
P2 → F2 / 1
P3 → — / 0

PHYSICAL RAM

F0 P0
F1 P1
F2 P2
F3 P3

Disk: P3

저장장치에서 P3를 F3으로 이동

만약 RAM에 빈 프레임이 있다면
저장장치에 저장되어 있던 프레임을 램에 적재 (Page Load)

Page Fault를 해결하는 과정

VIRTUAL

P0
P1
P2
P3

PAGE TABLE

P0 → F0 / 1
P1 → F1 / 1
P2 → F2 / 1
P3 → F3 / 1

PHYSICAL RAM

F0 P0
F1 P1
F2 P2
F3 P3

Disk: P3

P3 → F3, Present=1 → Instruction Restart

이후 Page Table을 업데이트한 RAM에 맞게 수정

User Mode로 다시 전환되어 명령 재실행

만약 RAM에 빈 프레임이 없다면?

VIRTUAL

새로 필요한 P3

Free Frame 없음

PHYSICAL RAM

F0 P0

F1 P1

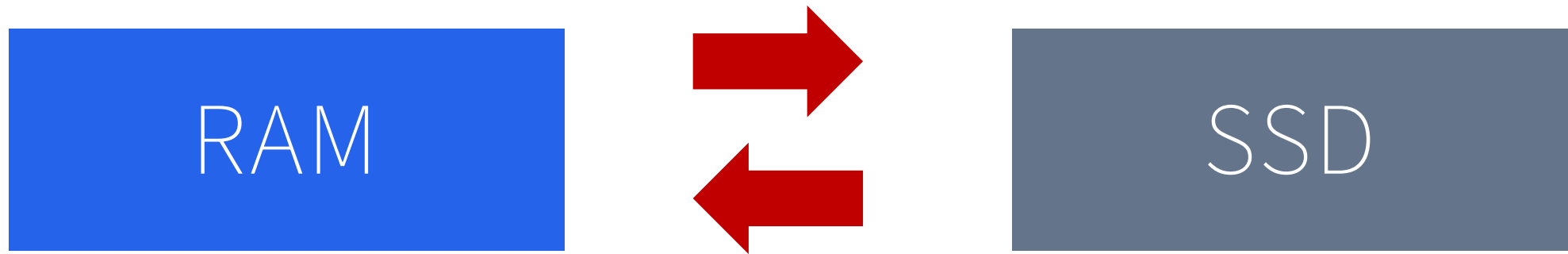
F2 P2

FIFO: 먼저 적재한 페이지를 없애고 그 자리에 새로운 프레임 적재

LRU: 오랫동안 사용하지 않은 페이지를 없애고 그 자리에 새로운 프레임 적재

(RAM이 부족하면 이것들이 너무 자주 발생해서 느려짐)

Backing Store & Swap

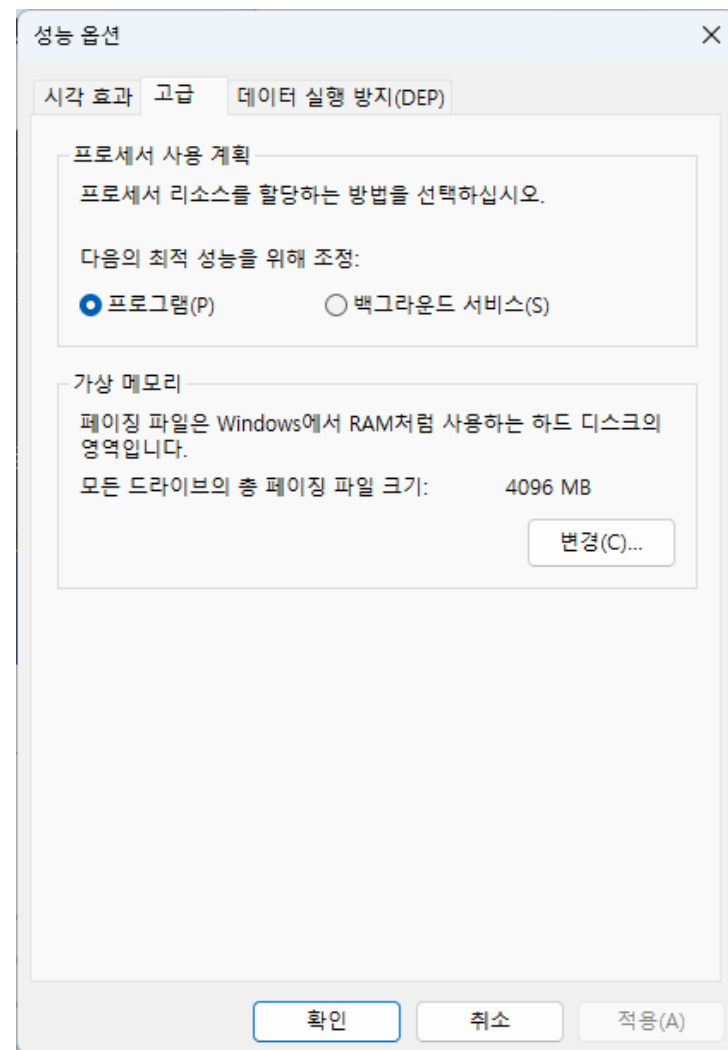
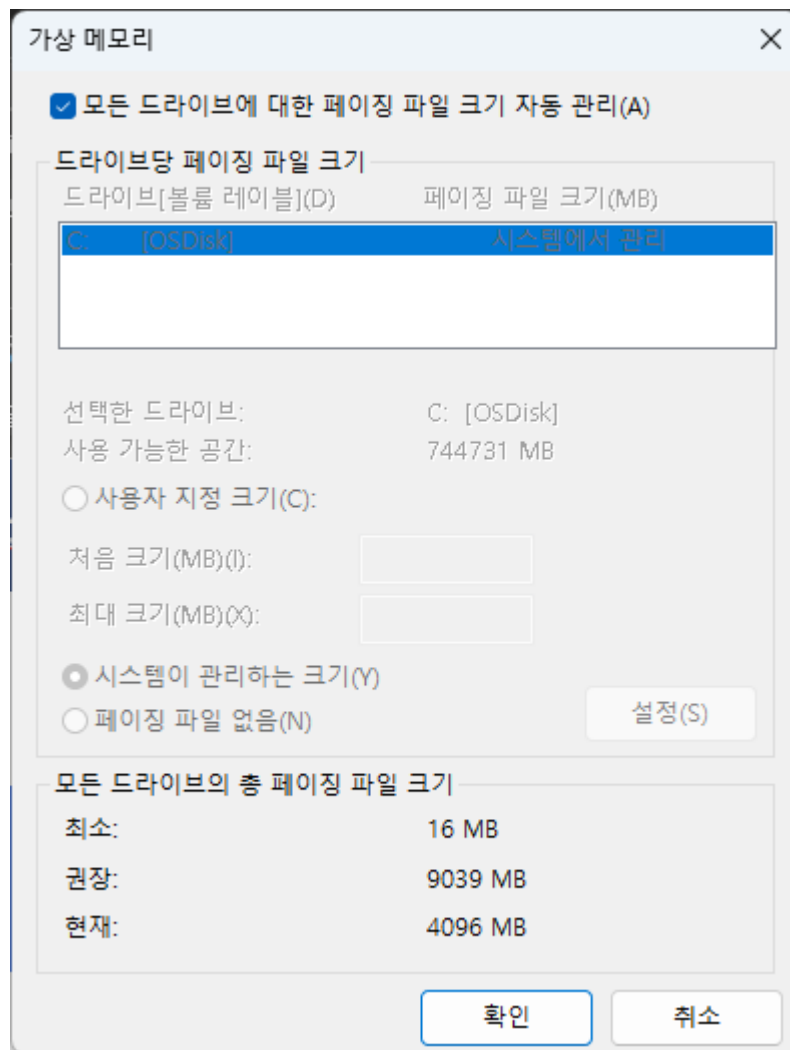


Backing Store: RAM에 없는 페이지를 보관할 수 있는 보조 저장공간

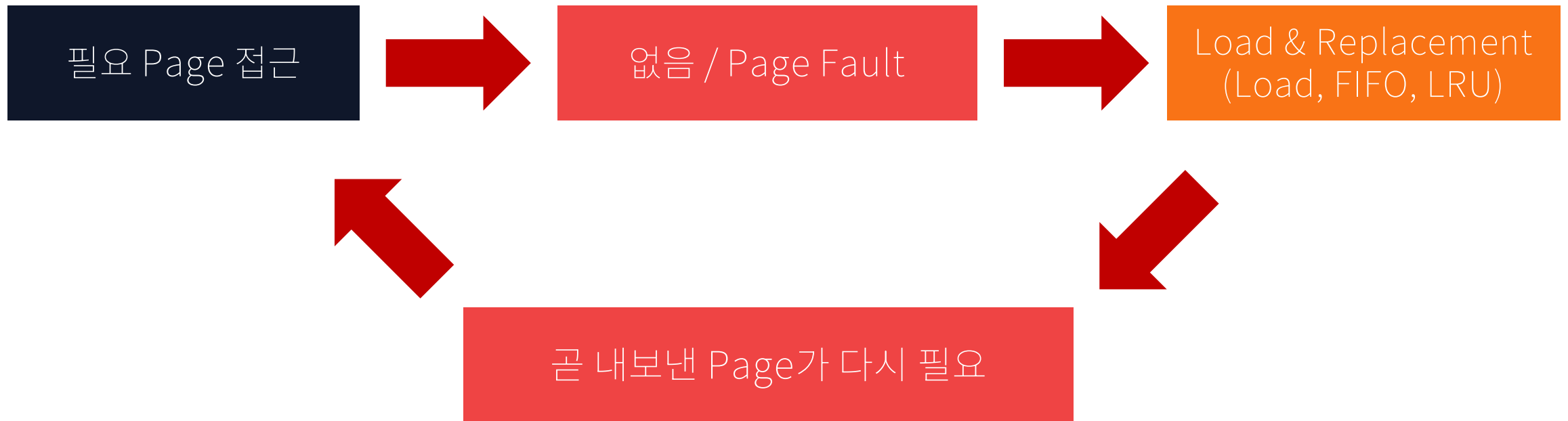
Swap: Backing Store에서 페이지를 내보내거나 가져오는 작업

Backing Store & Swap

윈도우에서는 SSD를 이용해
Backing Store를 구현하고
그 크기도 설정할 수 있음



Thrashing



RAM이 부족하면 Page Fault가 증가하게 되고, Swap / Page Replacement가 증가하게 됨
이 상태에서는 실제 작업보다 페이지 교체에 시간을 더 쓰게 되고, 성능이 급락함
이러한 상태를 Thrashing이라 함